

Lab 4: Limited Dependent Variable Models

SOCIOLOGY 723 — Social Statistics II — Thursday, September 17

Wenhao Jiang

Table of contents

1	Goals	1
2	Binary outcomes	2
2.1	Odds ratios	3
2.2	Average marginal effects	3
2.3	Predicted probabilities, the observed-value way	4
2.4	Robust and clustered standard errors	5
3	Interactions	6
4	Separation	7
5	Count outcomes	7
5.1	Is it overdispersed?	8
5.2	Does the model reproduce the data?	9
6	Ordered outcomes	10
6.1	Testing proportional odds	11
7	Model fit	12
8	Exercises	13
9	Session information	13

1 Goals

1. Fit and read logit and probit models, and see how little the choice matters.
2. Interpret on the probability scale: average marginal effects and predicted probabilities, with intervals.
3. Handle interactions correctly in a nonlinear model.
4. Diagnose and repair separation.

5. Fit count models, detect overdispersion, and choose a remedy.
6. Fit an ordered logit and test proportional odds.

```
library(dplyr)
library(ggplot2)
library(marginaleffects)
library(sandwich); library(lmtest)
library(MASS)      # glm.nb, polr
library(AER)       # dispersiontest
library(brglm2)    # Firth penalized likelihood
library(pROC)

gss <- readRDS("../Data/gss_earnings.rds")
```

2 Binary outcomes

Outcome: `ba`, whether the respondent holds a bachelor's degree or more.

```
f <- ba ~ pareduc + wordsum + female + black + exper

m_lpm    <- lm(f, data = gss)
m_logit  <- glm(f, data = gss, family = binomial(link = "logit"))
m_probit <- glm(f, data = gss, family = binomial(link = "probit"))

round(cbind(LPM = coef(m_lpm),
            logit = coef(m_logit),
            probit = coef(m_probit),
            ratio = coef(m_logit) / coef(m_probit)), 4)

#>           LPM    logit  probit ratio
#> (Intercept) -0.3241 -4.8700 -2.7988 1.740
#> pareduc      0.0297  0.1798  0.1014 1.774
#> wordsum      0.0740  0.4125  0.2448 1.685
#> female       0.0478  0.2780  0.1581 1.759
#> black       -0.0421 -0.1510 -0.0880 1.715
#> exper       -0.0061 -0.0313 -0.0190 1.647
```

The logit-to-probit ratio hovers around 1.7, as the theory predicts. The two are *different* models — different link functions, different assumed error distributions — but they are close enough empirically that the choice between them almost never matters for the quantities we report.

```
## how often does the LPM predict outside [0,1]?
mean(fitted(m_lpm) < 0 | fitted(m_lpm) > 1)
#> [1] 0.04645
range(fitted(m_lpm))
```

```
#> [1] -0.5137 1.0268
```

i Note

The linear probability model is not automatically wrong — but check this number. If a substantial share of fitted values leaves $[0, 1]$, the linear approximation is being asked to do something it cannot.

2.1 Odds ratios

```
round(exp(cbind(OR = coef(m_logit), confint.default(m_logit))), 3)
#>
#> OR 2.5 % 97.5 %
#> (Intercept) 0.008 0.005 0.013
#> pareduc 1.197 1.165 1.230
#> wordsum 1.511 1.440 1.585
#> female 1.320 1.132 1.540
#> black 0.860 0.692 1.069
#> exper 0.969 0.962 0.976
```

Read the **wordsum** row as: each additional correct word multiplies the *odds* of holding a BA by about 1.5. Whether that corresponds to a big or small change in the *probability* depends entirely on where you start — which is why we now move to the probability scale.

2.2 Average marginal effects

```
avg_slopes(m_logit)
#>
#> Term Contrast Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
#> black 1 - 0 -0.02824 0.020682 -1.37 0.172 2.5 -0.06877 0.01230
#> exper dY/dX -0.00588 0.000663 -8.88 <0.001 60.3 -0.00718 -0.00458
#> female 1 - 0 0.05224 0.014702 3.55 <0.001 11.4 0.02343 0.08105
#> pareduc dY/dX 0.03375 0.002342 14.41 <0.001 154.0 0.02916 0.03834
#> wordsum dY/dX 0.07743 0.003912 19.79 <0.001 287.3 0.06976 0.08510
#>
#> Type: response
```

Compare with the marginal effect at the mean, and with the LPM coefficients:

```
mem <- slopes(m_logit, newdata = "mean")
comparison <- data.frame(
  variable = c("pareduc", "wordsum", "female", "black", "exper"),
  AME = avg_slopes(m_logit)$estimate,
  MEM = mem$estimate,
  LPM = coef(m_lpm)[-1]
```

```
)
round(comparison[, -1], 4) |> `rownames<-`(comparison$variable)
#>           AME      MEM      LPM
#> pareduc -0.0282 -0.0316  0.0297
#> wordsum -0.0059 -0.0068  0.0740
#> female   0.0522  0.0628  0.0478
#> black    0.0338  0.0388 -0.0421
#> exper    0.0774  0.0890 -0.0061
```

Three observations worth taking seriously:

- The AME and the LPM coefficients are very close. This is typical, and it is the main reason people defend the LPM for descriptive and causal work.
- The MEM differs from the AME. It answers a different question: the effect for one hypothetical person at the mean of everything, including means of dummy variables.
- Following Hanmer and Kalkan (2013), report the **AME**.

2.3 Predicted probabilities, the observed-value way

```
avg_predictions(m_logit, variables = list(wordsum = c(3, 6, 9)))
#>
#> wordsum Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>      3    0.159    0.01209 13.1  <0.001 128.6 0.135 0.183
#>      6    0.366    0.00855 42.8  <0.001   Inf 0.349 0.383
#>      9    0.631    0.01467 43.0  <0.001   Inf 0.602 0.659
#>
#> Type: response
```

This sets `wordsum` to each value for *every* respondent, leaving their other covariates alone, computes predicted probabilities, and averages. Contrast with setting everything else to its mean:

```
predictions(m_logit, newdata = datagrid(wordsum = c(3, 6, 9)))
#>
#> wordsum Estimate Pr(>|z|)      S 2.5 % 97.5 %
#>      3    0.118  <0.001 256.7 0.0974 0.141
#>      6    0.315  <0.001 124.8 0.2900 0.341
#>      9    0.613  <0.001  26.4 0.5749 0.650
#>
#> Type: invlink(link)
```

```
plot_predictions(m_logit, by = "wordsum") +
  labs(x = "Vocabulary score (0-10)", y = "Pr(bachelor's or more)") +
  theme_minimal(base_size = 10)
```

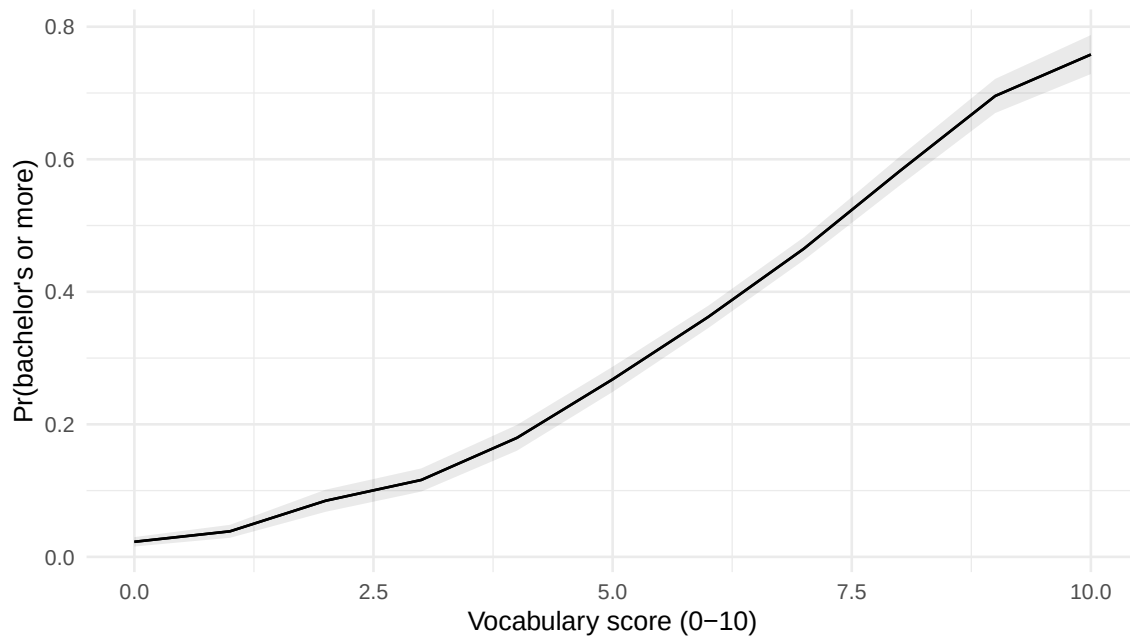


Figure 1: Average predicted probability of holding a BA by vocabulary score (observed-value approach), with 95% intervals.

And the contrast we probably want to report:

```
avg_comparisons(m_logit, variables = list(wordsum = c(3, 9)))
#>
#> Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
#> 0.472 0.0236 20 <0.001 292.4 0.425 0.518
#>
#> Term: wordsum
#> Type: response
#> Comparison: 9 - 3
```

2.4 Robust and clustered standard errors

Everything from Week 2 carries over — the sandwich works on `glm` objects.

```
round(cbind(
  classical = sqrt(diag(vcov(m_logit))),
  HCO       = sqrt(diag(vcovHC(m_logit, type = "HCO"))),
  cluster   = sqrt(diag(vcovCL(m_logit, cluster = ~ psu))),
), 4)
#>
#>      classical      HCO cluster
#> (Intercept)  0.2640 0.2842 0.2834
#> pareduc      0.0137 0.0157 0.0159
#> wordsum      0.0246 0.0252 0.0250
```

```
#> female      0.0785 0.0781  0.0735
#> black       0.1111 0.1119  0.1110
#> exper       0.0037 0.0036  0.0036
```

```
avg_slopes(m_logit, vcov = ~ psu)
```

```
#>
#>      Term Contrast Estimate Std. Error      z Pr(>|z|)      S    2.5 %    97.5 %
#> black      1 - 0 -0.02824   0.020651 -1.37    0.172     2.5 -0.06871  0.01224
#> exper      dY/dX -0.00588   0.000648 -9.07   <0.001    62.9 -0.00715 -0.00461
#> female      1 - 0  0.05224   0.013762  3.80   <0.001    12.7  0.02527  0.07921
#> pareduc     dY/dX  0.03375   0.002705 12.48   <0.001   116.3  0.02845  0.03905
#> wordsum     dY/dX  0.07743   0.004021 19.25   <0.001   272.0  0.06955  0.08531
#>
#> Type: response
```

3 Interactions

Does the vocabulary gradient differ by gender?

```
m_int <- glm(ba ~ pareduc + wordsum * female + black + exper,
             data = gss, family = binomial)
round(coef(summary(m_int))[, 1:2], 4)
#>
#>      Estimate Std. Error
#> (Intercept)  -4.4690    0.2983
#> pareduc       0.1800    0.0137
#> wordsum       0.3533    0.0323
#> female      -0.5842    0.3282
#> black       -0.1390    0.1115
#> exper       -0.0318    0.0037
#> wordsum:female 0.1306    0.0483
```

The product-term coefficient is on the **log-odds** scale. What we actually want is whether the *marginal effect on the probability* differs by gender:

```
avg_slopes(m_int, variables = "wordsum", by = "female")
#>
#> female Estimate Std. Error      z Pr(>|z|)      S    2.5 %    97.5 %
#>      0  0.0661    0.00524 12.6   <0.001  118.8  0.0558  0.0763
#>      1  0.0907    0.00548 16.5   <0.001  201.9  0.0800  0.1015
#>
#> Term: wordsum
#> Type: response
#> Comparison: dY/dX
```

```
avg_comparisons(m_int, variables = "wordsum", by = "female",
                 hypothesis = "b2 - b1 = 0")
#>
#> Hypothesis Estimate Std. Error      z Pr(>|z|)      S  2.5 % 97.5 %
#>      b2-b1=0   0.0241    0.00746  3.22  0.00127  9.6 0.00942 0.0387
#>
#> Type: response
```

! Important

Note the two can disagree in sign or significance (Ai and Norton 2003). In a nonlinear model the interaction on the probability scale depends on where each person sits on the curve, not only on the product term. **Always report the quantity you actually mean, with an interval.**

4 Separation

Construct a variable that perfectly predicts the outcome, and watch the MLE run off to infinity.

```
gss2 <- gss |> mutate(perfect = as.integer(educ >= 18 & ba == 1))
m_sep <- glm(ba ~ pareduc + perfect, data = gss2, family = binomial)
round(coef(summary(m_sep))[, 1:2], 3)
#>
#> Estimate Std. Error
#> (Intercept) -3.899    0.207
#> pareduc      0.232    0.015
#> perfect      19.320   271.710
```

The `perfect` coefficient is enormous with an enormous standard error — the classic signature. The fix is Firth's penalized likelihood:

```
m_firth <- glm(ba ~ pareduc + perfect, data = gss2,
               family = binomial, method = "brglmFit")
round(coef(summary(m_firth))[, 1:2], 3)
#>
#> Estimate Std. Error
#> (Intercept) -3.893    0.207
#> pareduc      0.232    0.015
#> perfect       7.803    1.424
```

Finite estimates, usable standard errors, and the variable is retained (Zorn 2005).

5 Count outcomes

Outcome: `childs`, the number of children.

```

m_pois <- glm(childs ~ educ + female + black + exper + evermar,
              data = gss, family = poisson(link = "log"))
round(coef(summary(m_pois))[, 1:2], 4)
#>               Estimate Std. Error
#> (Intercept)  -0.0064      0.0876
#> educ         -0.0455      0.0048
#> female        0.0089      0.0269
#> black         0.3600      0.0349
#> exper         0.0118      0.0013
#> evermar       0.9468      0.0430

```

Coefficients are semi-elasticities: e^β is the multiplicative change in the expected count.

```

round(exp(coef(m_pois)), 3)
#> (Intercept)      educ      female      black      exper      evermar
#>      0.994      0.956      1.009      1.433      1.012      2.578

```

5.1 Is it overdispersed?

```

c(mean = mean(gss$childs), var = var(gss$childs))
#> mean var
#> 1.604 2.109

dispersiontest(m_pois, trafo = 1)
#>
#> Overdispersion test
#>
#> data: m_pois
#> z = 4, p-value = 0.00004
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#> alpha
#> 0.1676

```

```

m_qp <- glm(childs ~ educ + female + black + exper + evermar,
            data = gss, family = quasipoisson)
m_nb <- glm.nb(childs ~ educ + female + black + exper + evermar, data = gss)

round(cbind(
  poisson_se = sqrt(diag(vcov(m_pois))),
  quasi_se   = sqrt(diag(vcov(m_qp))),
  negbin_se  = sqrt(diag(vcov(m_nb)))[names(coef(m_pois))],
  poisson_robust = sqrt(diag(vcovHC(m_pois, type = "HC0")))
), 4)

```



```
#>      poisson_se quasi_se negbin_se poisson_robust
#> (Intercept)    0.0876   0.0939    0.0900         0.1012
#> educ          0.0048   0.0051    0.0049         0.0052
#> female        0.0269   0.0288    0.0277         0.0278
#> black         0.0349   0.0374    0.0361         0.0373
#> exper         0.0013   0.0014    0.0014         0.0013
#> evermar       0.0430   0.0461    0.0437         0.0551
```

Note that the coefficients barely move but the standard errors do. That is the signature of overdispersion: the mean is fine, the variance assumption is not.

```
c(theta = m_nb$theta, se_theta = m_nb$SE.theta)
#>      theta se_theta
#>      31.25   14.91
AIC(m_pois, m_nb)
#>      df    AIC
#> m_pois  6 10778
#> m_nb    7 10775
```

A finite $\hat{\theta}$ well away from infinity, and a much lower AIC, both say the negative binomial is doing real work.

5.2 Does the model reproduce the data?

```
set.seed(1)
n <- nrow(gss)
sim <- bind_rows(
  tibble::tibble(k = gss$chlds, source = "observed"),
  tibble::tibble(k = rpois(n, fitted(m_pois)), source = "Poisson"),
  tibble::tibble(k = rnbinom(n, mu = fitted(m_nb), size = m_nb$theta),
                 source = "Neg. binomial")
) |>
  count(source, k) |>
  group_by(source) |> mutate(p = n / sum(n)) |> ungroup() |> # normalize on FULL n
  filter(k <= 8) # then trim for display

ggplot(sim, aes(k, p, fill = source)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("observed" = "grey30", "Poisson" = "navy",
                              "Neg. binomial" = "firebrick")) +
  labs(x = "number of children", y = "proportion", fill = NULL) +
  theme_minimal(base_size = 10)
```

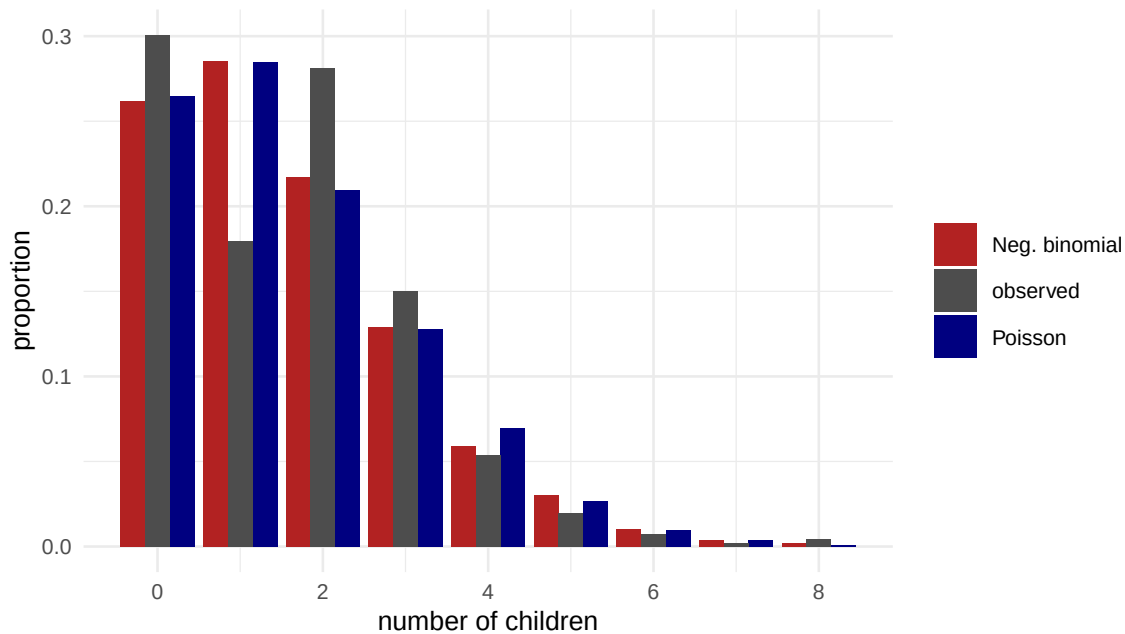


Figure 2: Observed distribution of the number of children against the distributions implied by the Poisson and negative binomial fits.

This *predictive check* is the single most informative count diagnostic, and the one people most often skip.

6 Ordered outcomes

```
m_ord <- polr(degree ~ pareduc + wordsum + female + black,
              data = gss, Hess = TRUE)
round(coef(summary(m_ord)), 4)
#>
#> Value Std. Error t value
#> pareduc      0.2082      0.0104 19.9296
#> wordsum      0.3578      0.0192 18.5908
#> female      0.3349      0.0646  5.1829
#> black     -0.0238      0.0914 -0.2607
#> < HS|HS      1.8372      0.1616 11.3667
#> HS|Junior college  5.2081      0.1788 29.1209
#> Junior college|Bachelor  5.6724      0.1823 31.1134
#> Bachelor|Graduate  7.3582      0.1973 37.3032
```

`polr()` does not print *p*-values; get them from the *t*-values, or better, work on the probability scale:

```
avg_predictions(m_ord, variables = list(wordsum = c(3, 9)), type = "probs")
#>
#> Group wordsum Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
```

```

#> < HS          3  0.1265    0.00888 14.25    <0.001 150.7 0.1091 0.1439
#> < HS          9  0.0190    0.00195  9.72    <0.001  71.8 0.0152 0.0228
#> HS           3  0.6228    0.01180 52.80    <0.001   Inf 0.5997 0.6459
#> HS           9  0.2839    0.01091 26.02    <0.001 493.4 0.2625 0.3053
#> Junior college 3  0.0719    0.00446 16.12    <0.001 191.8 0.0632 0.0807
#> Junior college 9  0.0936    0.00506 18.49    <0.001 251.2 0.0837 0.1035
#> Bachelor       3  0.1373    0.00812 16.90    <0.001 210.5 0.1214 0.1532
#> Bachelor       9  0.3534    0.00993 35.58    <0.001 918.6 0.3339 0.3729
#> Graduate       3  0.0414    0.00367 11.27    <0.001  95.5 0.0342 0.0486
#> Graduate       9  0.2501    0.01092 22.91    <0.001 383.6 0.2287 0.2715
#>
#> Type: probs

```

6.1 Testing proportional odds

The model imposes one β across all cutpoints. Compare it to a multinomial logit, which frees the coefficients:

The proportional-odds model is *nested inside* a generalized ordered logit that lets each cutpoint have its own coefficients, so we can test the restriction by a likelihood-ratio test against that model.

```

library(VGAM)
## proportional odds imposed
m_po <- vglm(degree ~ pareduc + wordsum + female + black,
             family = cumulative(parallel = TRUE), data = gss)
## proportional odds relaxed
m_npo <- vglm(degree ~ pareduc + wordsum + female + black,
              family = cumulative(parallel = FALSE), data = gss)

lr <- 2 * (logLik(m_npo) - logLik(m_po))
df <- length(coef(m_npo)) - length(coef(m_po))
c(LR = lr, df = df, p = pchisq(lr, df, lower.tail = FALSE))
#> LR df p
#> NaN 12 NaN

```

A small p -value says the proportional-odds restriction is rejected. With n this large that is common; look at *how much* the free coefficients differ before abandoning the parsimony. (An ordered logit and a *multinomial* logit are not nested, so their deviances cannot be compared this way.)

7 Model fit

```
AIC(m_logit, m_int)
#>           df   AIC
#> m_logit    6 3896
#> m_int      7 3890
BIC(m_logit, m_int)
#>           df   BIC
#> m_logit    6 3933
#> m_int      7 3934

## McFadden's pseudo-R2
m_null <- glm(ba ~ 1, data = gss, family = binomial)
1 - as.numeric(logLik(m_logit)) / as.numeric(logLik(m_null))
#> [1] 0.1827
```

```
roc_obj <- roc(gss$ba, fitted(m_logit), quiet = TRUE)
plot(roc_obj, col = "navy", lwd = 2, legacy.axes = TRUE)
text(0.6, 0.2, paste("AUC =", round(auc(roc_obj), 3)))
```

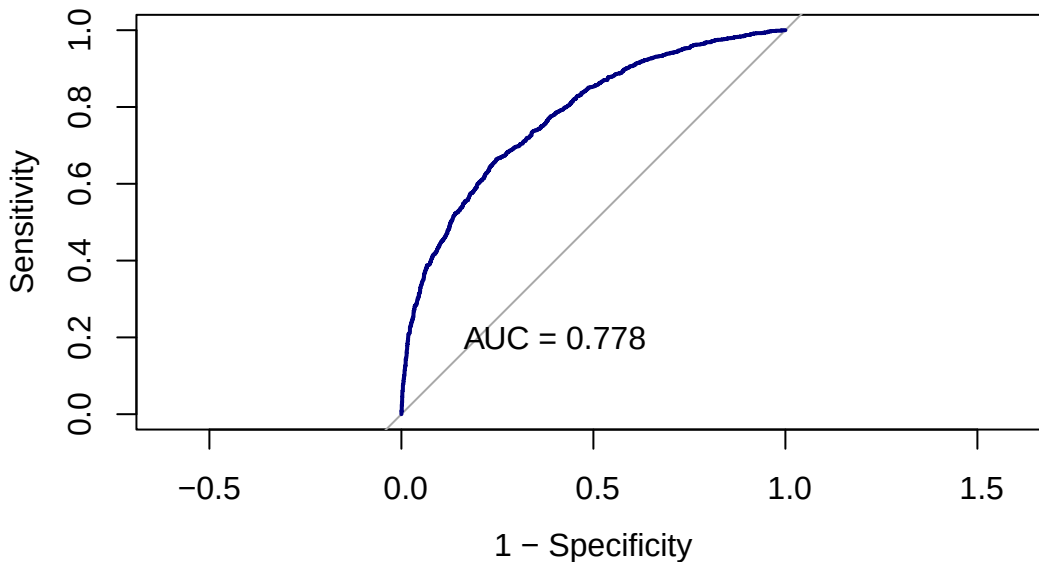


Figure 3: ROC curve for the logit model of holding a bachelor's degree.

```
## classification accuracy against the null rule of "always predict the majority"
pred <- as.integer(fitted(m_logit) > 0.5)
c(model = mean(pred == gss$ba),
  always_majority = max(mean(gss$ba), 1 - mean(gss$ba)))
#>           model always_majority
#>           0.7170           0.5893
```

8 Exercises

1. **Logit versus LPM for a causal contrast.** Estimate the “effect” of being ever-married (`evermar`) on holding a BA, controlling for `pareduc`, `wordsum`, `female`, `black`. Compare the LPM coefficient with the logit AME. How close are they? Which would you report, and to whom?
2. **The rescaling problem.** Fit the logit for `ba` with (a) `pareduc` only and (b) `pareduc + wordsum + female + black`. Compare the `pareduc coefficient` across the two, then compare the `pareduc AME`. Explain the difference using Mood (2010).
3. **Simulation-based intervals.** Take the logit model and get an interval for the difference in predicted probability between `wordsum = 3` and `wordsum = 9` by simulating 10,000 draws of $\tilde{\beta}$ from $\mathcal{N}(\hat{\beta}, \widehat{\text{Var}}(\hat{\beta}))$ (King, Tomz, and Wittenberg 2000). Compare to the delta-method interval from `avg_comparisons()`.
4. **Offsets.** Suppose you wanted to model children *per year since age 18* rather than children. Add `offset(log(pmax(age - 18, 1)))` to the Poisson and interpret the coefficients. What changed?
5. **Zero inflation.** Fit `pscl::zeroinfl(childs ~ ... | evermar + age)` and compare with the negative binomial by AIC and by the predictive check above. Does the zero-inflation story make substantive sense here?
6. **Separation in the wild.** Fit a logit for `ba` including `factor(region) * black`. Does anything blow up? If so, apply Firth and report what changes.

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
```

```

#> [1] splines      stats4      stats      graphics  grDevices  utils      datasets
#> [8] methods      base
#>
#> other attached packages:
#> [1] VGAM_1.1-13      pROC_1.19.0.1      brglm2_1.0.0
#> [4] AER_1.2-15      survival_3.6-4      car_3.1-3
#> [7] carData_3.0-5    MASS_7.3-60.2      lmtest_0.9-40
#> [10] zoo_1.8-12      sandwich_3.1-1      marginaleffects_0.32.0
#> [13] ggplot2_4.0.0    dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] utf8_1.2.4      generics_0.1.3      enrichwith_0.4.0
#> [4] lattice_0.22-7  digest_0.6.37      magrittr_2.0.3
#> [7] evaluate_1.0.0  grid_4.4.1          RColorBrewer_1.1-3
#> [10] nleqslv_3.3.5   fastmap_1.2.0      jsonlite_1.8.9
#> [13] Matrix_1.7-0    nnet_7.3-19         backports_1.5.0
#> [16] Formula_1.2-5   tinytex_0.53        fansi_1.0.6
#> [19] scales_1.4.0    numDeriv_2016.8-1.1 abind_1.4-8
#> [22] cli_3.6.5       rlang_1.3.0         withr_3.0.1
#> [25] yaml_2.3.10     tools_4.4.1         checkmate_2.3.2
#> [28] vctrs_0.6.5     R6_2.5.1            lifecycle_1.0.4
#> [31] insight_1.4.6   pkgconfig_2.0.3     pillar_1.9.0
#> [34] gtable_0.3.6    Rcpp_1.1.1          data.table_1.18.2.1
#> [37] glue_1.7.0      statmod_1.5.0       xfun_0.47
#> [40] tibble_3.2.1    tidyselect_1.2.1    knitr_1.48
#> [43] farver_2.1.2    htmltools_0.5.8.1   labeling_0.4.3
#> [46] rmarkdown_2.28  compiler_4.4.1      S7_0.2.0

```